Balanced Binary Search Trees and std::Set

DAVID HAN DLHAN2@ILLINOIS.EDU

What is a set?

- Set (math) - **unordered** collection of **unique** objects
- std::set (C++) - **ordered** collection of **unique** objects

	Ordered	Unordered
No Duplicates	std::set	std::unordered_set
Duplicates	std::multiset	std::unordered_multiset

- Under the hood – std::set is implemented as a balanced binary search tree (BST)

Binary Search Tree

- A rooted tree where each node has ≤ 2 children
- Each node maintains a value and pointers to left and right child
- For any node x:
  - Value of left child < value of x
  - Value of right child > value of x

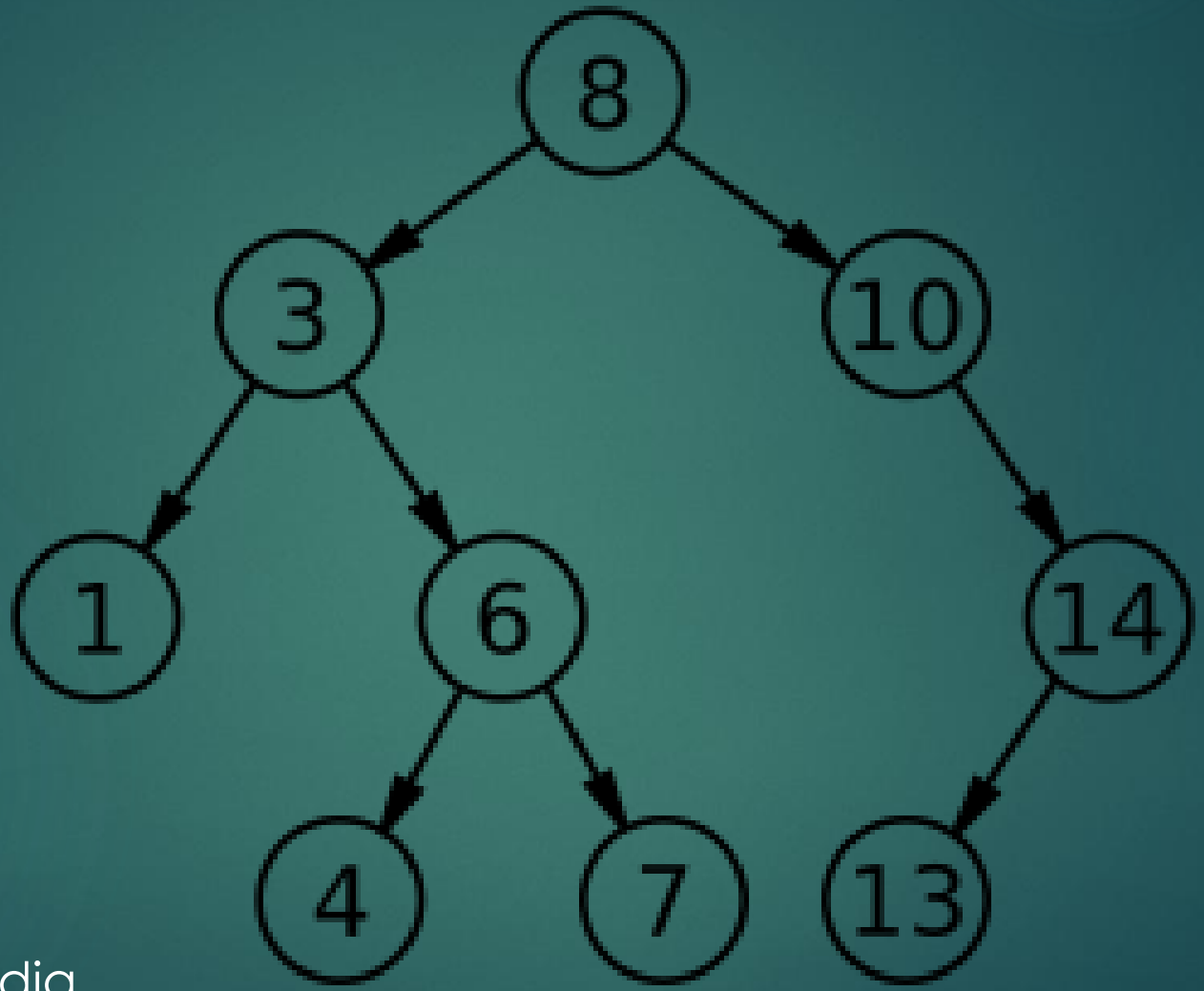
Operations Supported

- Insert
- Erase
- Find
- Begin
- Lower bound
- Size

Set::find(T x)

- Start from the root
- Repeatedly traverse down the tree
  - If x == current node value, found!
  - If x < current node value, go to left child
  - If x > current node value, go to right child
- If at any point we cannot go further but haven't found x, x doesn't exist in our BST. (std::set::find returns set::end() iterator)
- Otherwise, we return iterator to the node found

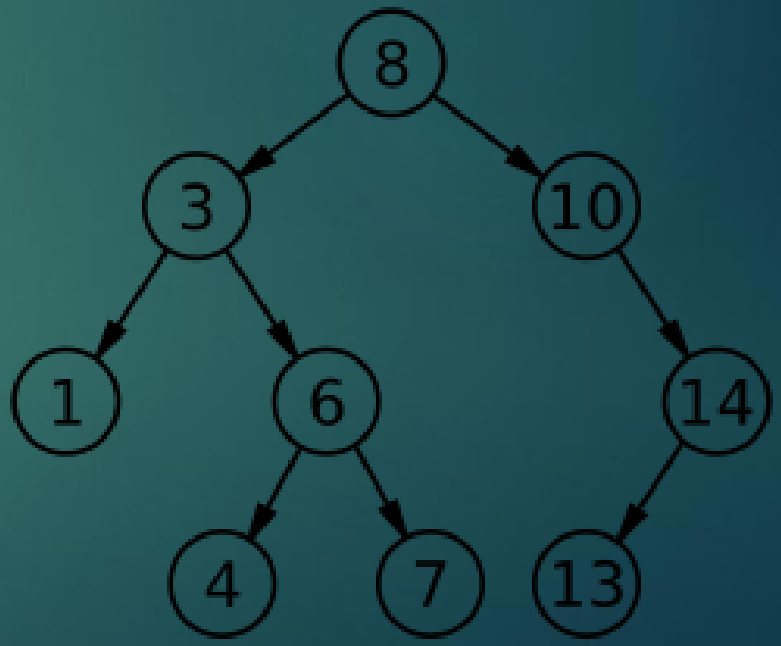
Find 7



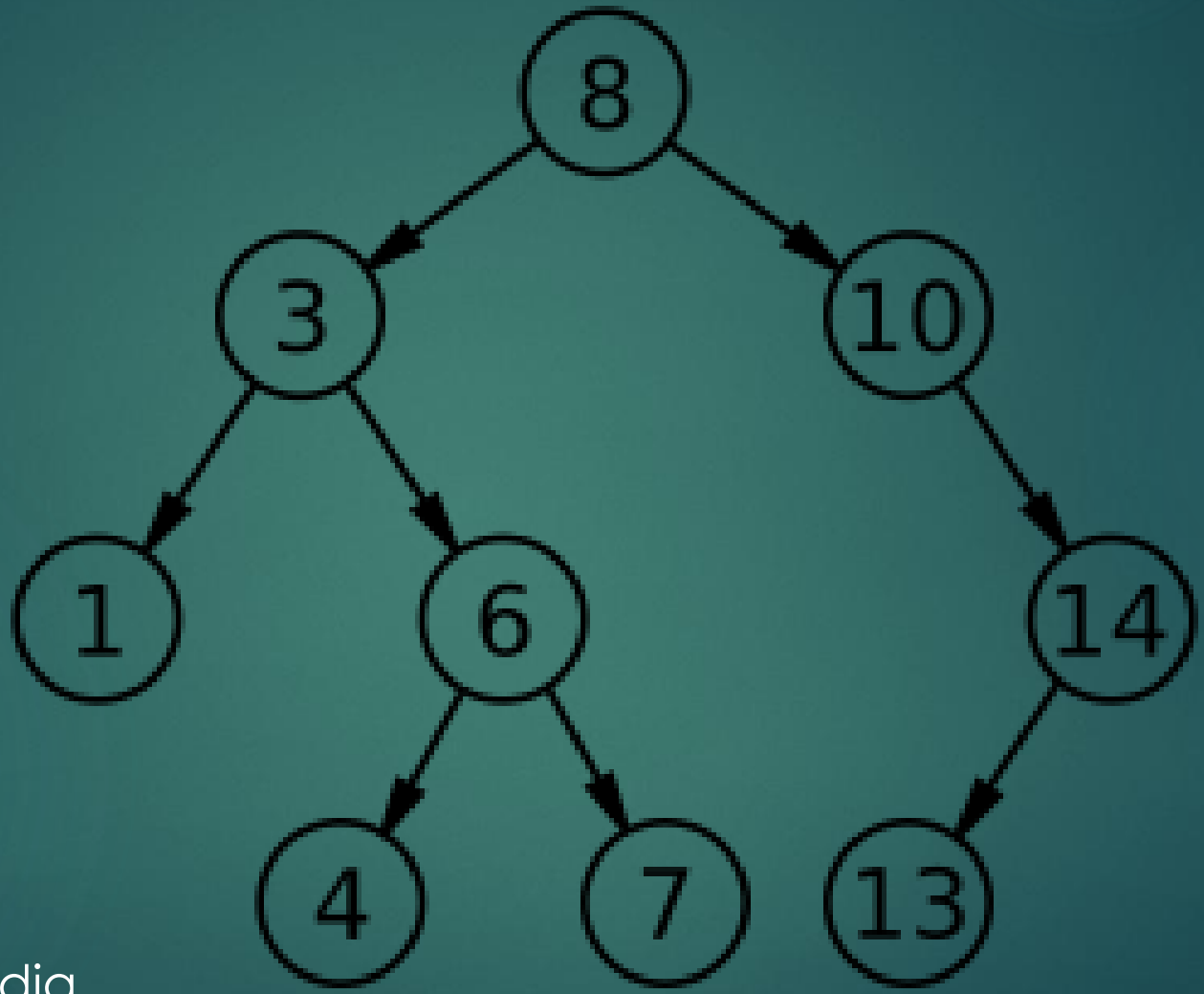
- Source: Wikipedia

Set::insert(T x)

- Start from the root
- Repeatedly traverse down the tree
  - If x == current node value, we are done (no inserts)!
  - If x < current node value, go to left child
  - If x > current node value, go to right child



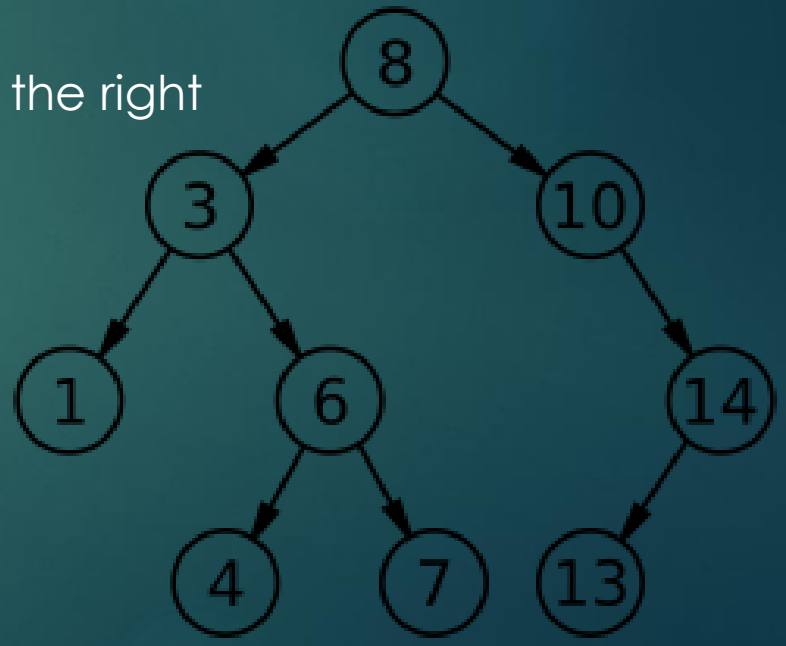
Insert 2



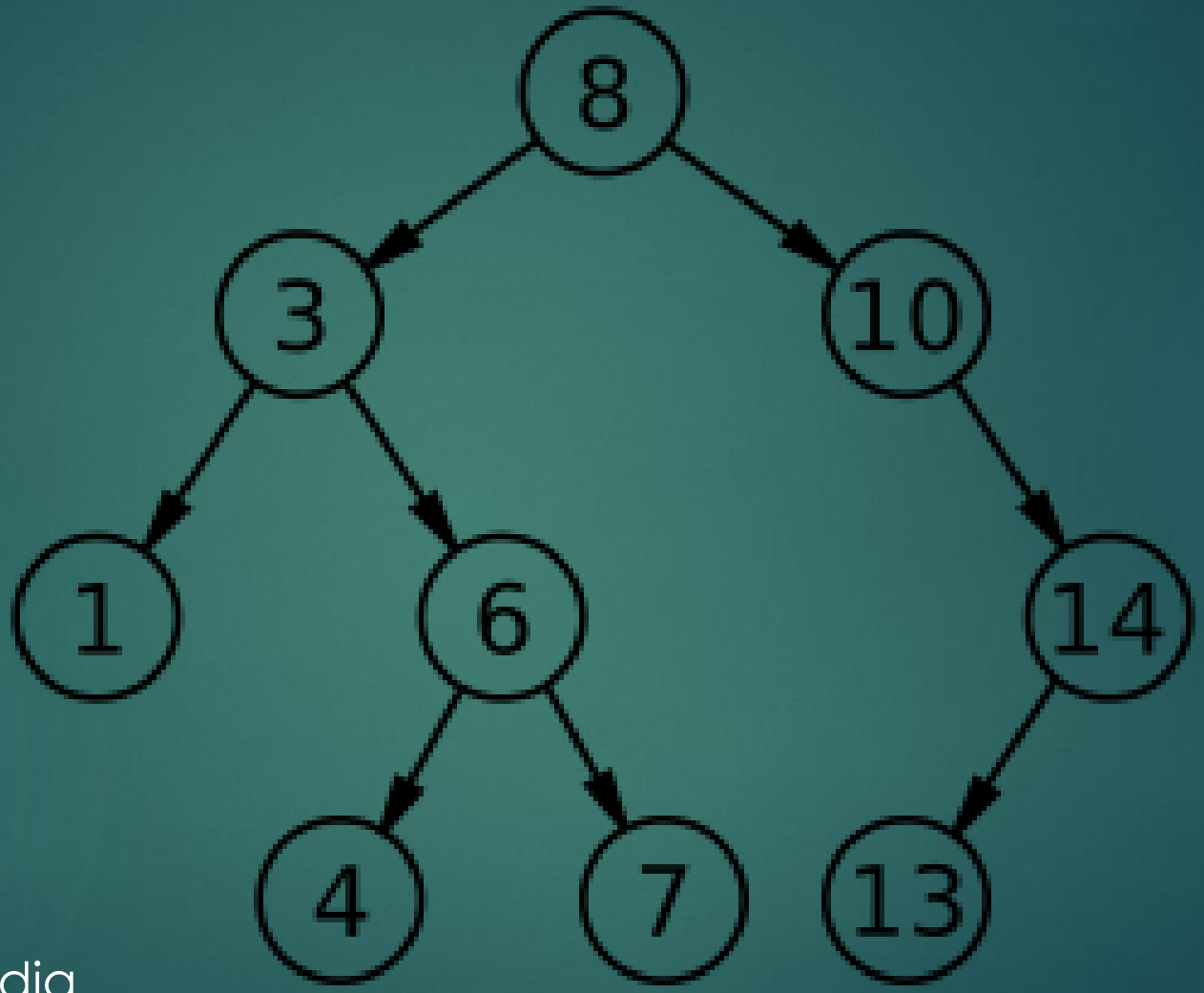
- Source: Wikipedia

Set::erase(T x)

- Find the node
  - If node is a leaf, just erase it
  - If it has one child, parent node now points to its child and we delete the node itself
  - If it has two children, swap it with the smallest element in the right subtree (go right, then repeatedly go left)

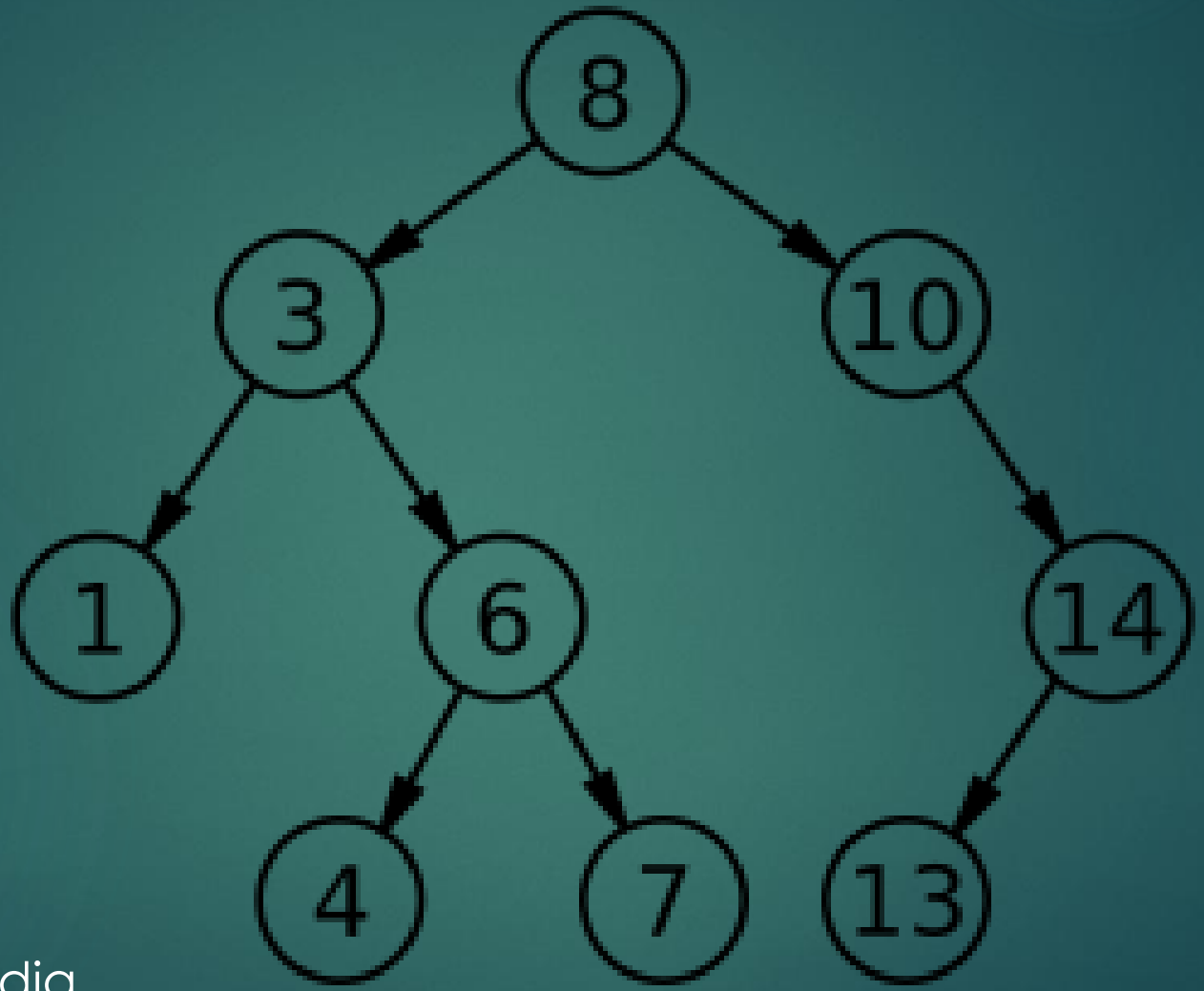


Erase 1



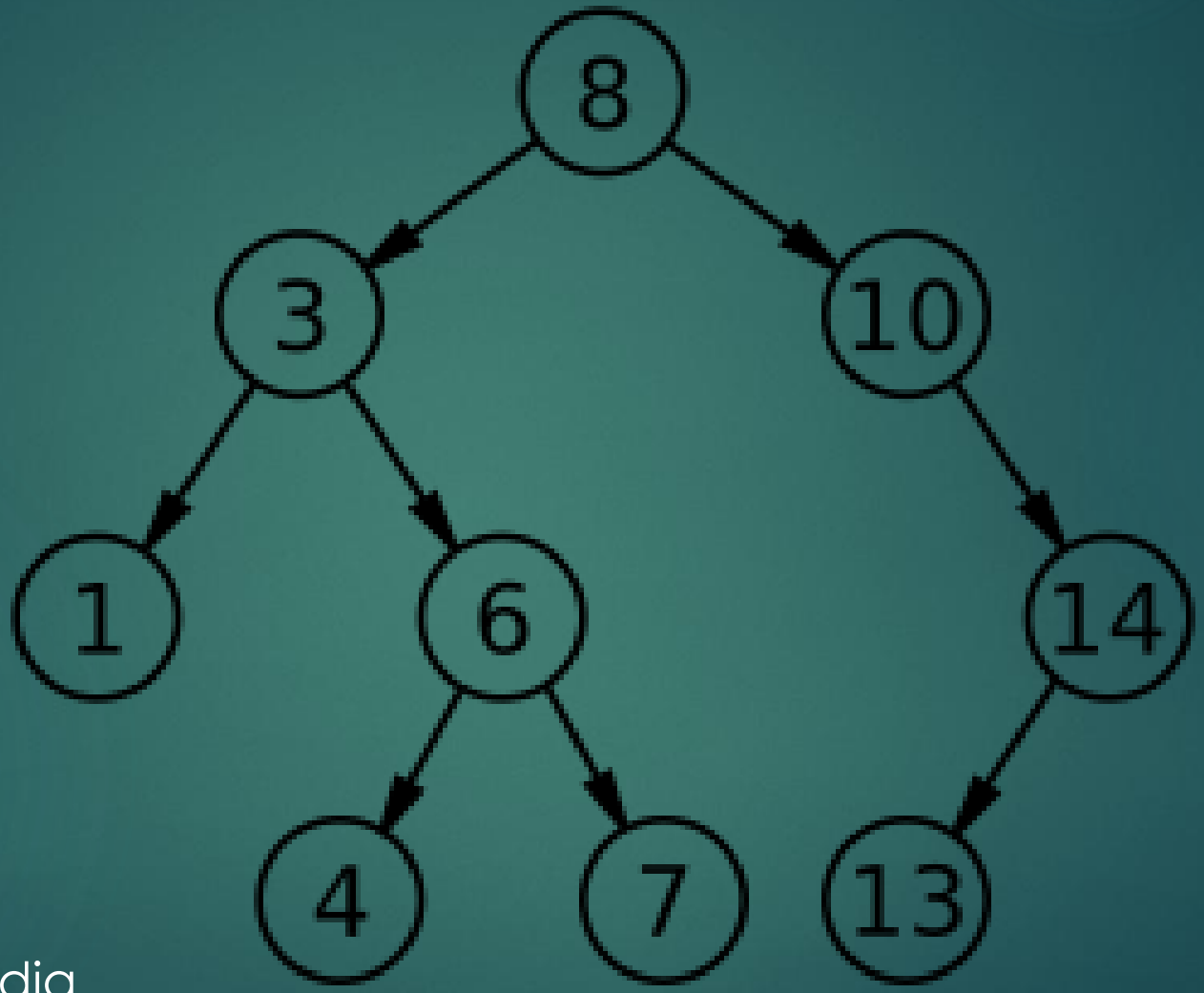
- Source: Wikipedia

Erase 10



- Source: Wikipedia

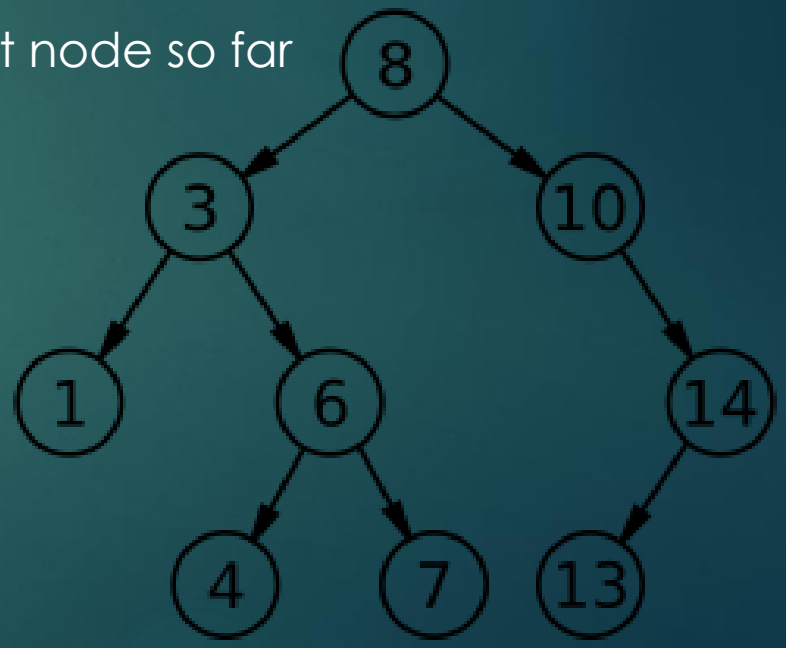
Erase 3



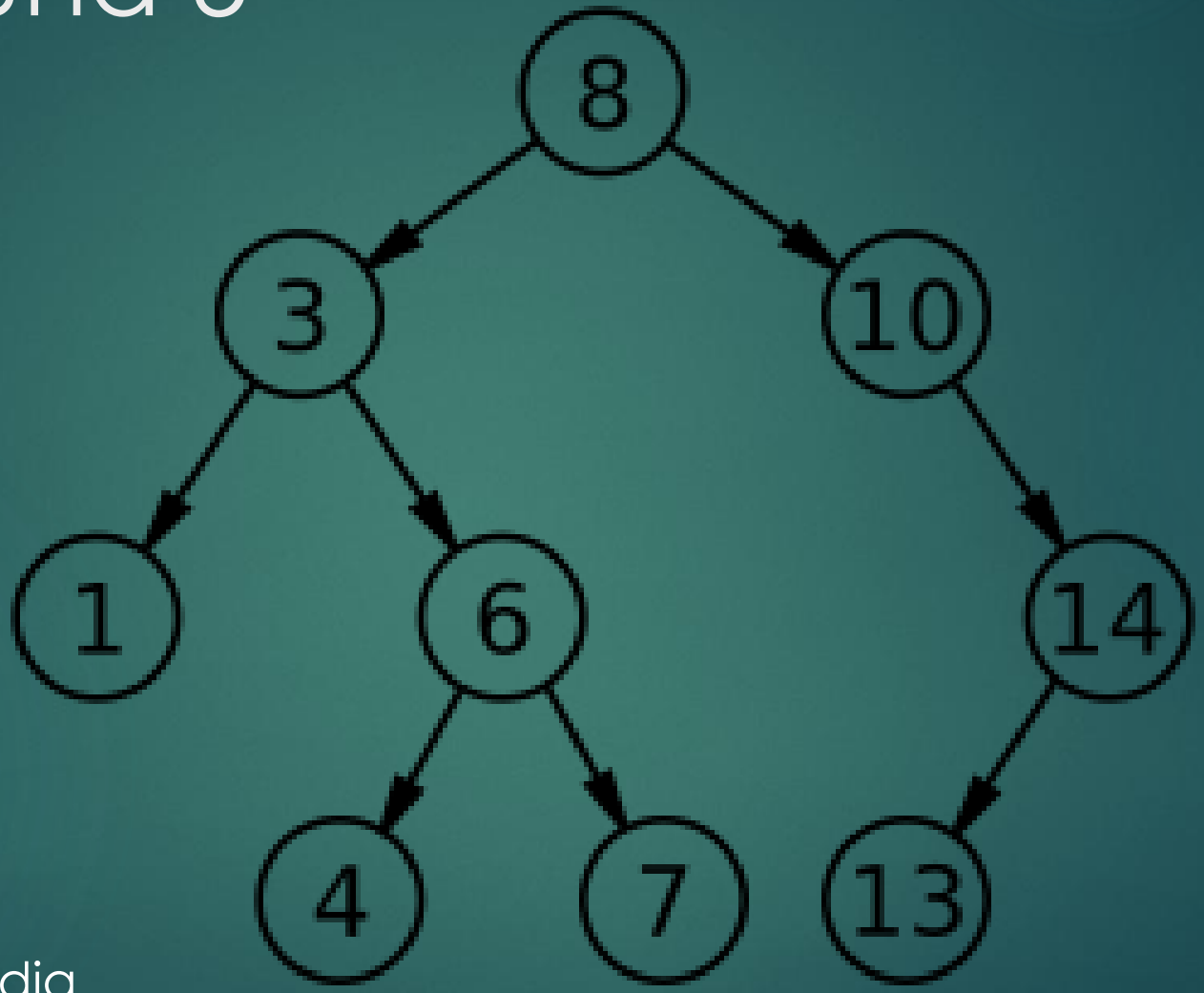
- Source: Wikipedia

Set::lower_bound(T x)

- Find smallest node ≥ x.
- Start from the root
- Repeatedly traverse down the tree
  - If x ≤ current node value, set current node as the smallest node so far and go to left child
  - Else, go to right child
- Return smallest node



Lower bound 5



- Source: Wikipedia

BST worst case

- Suppose we inserted numbers 1, 2, 3, ..., n in that order.
  - What does our BST look like?
  - What is the runtime for find/insert/delete?
- Our BST becomes a linked list :(

Solution to BST worst case - Balanced BST

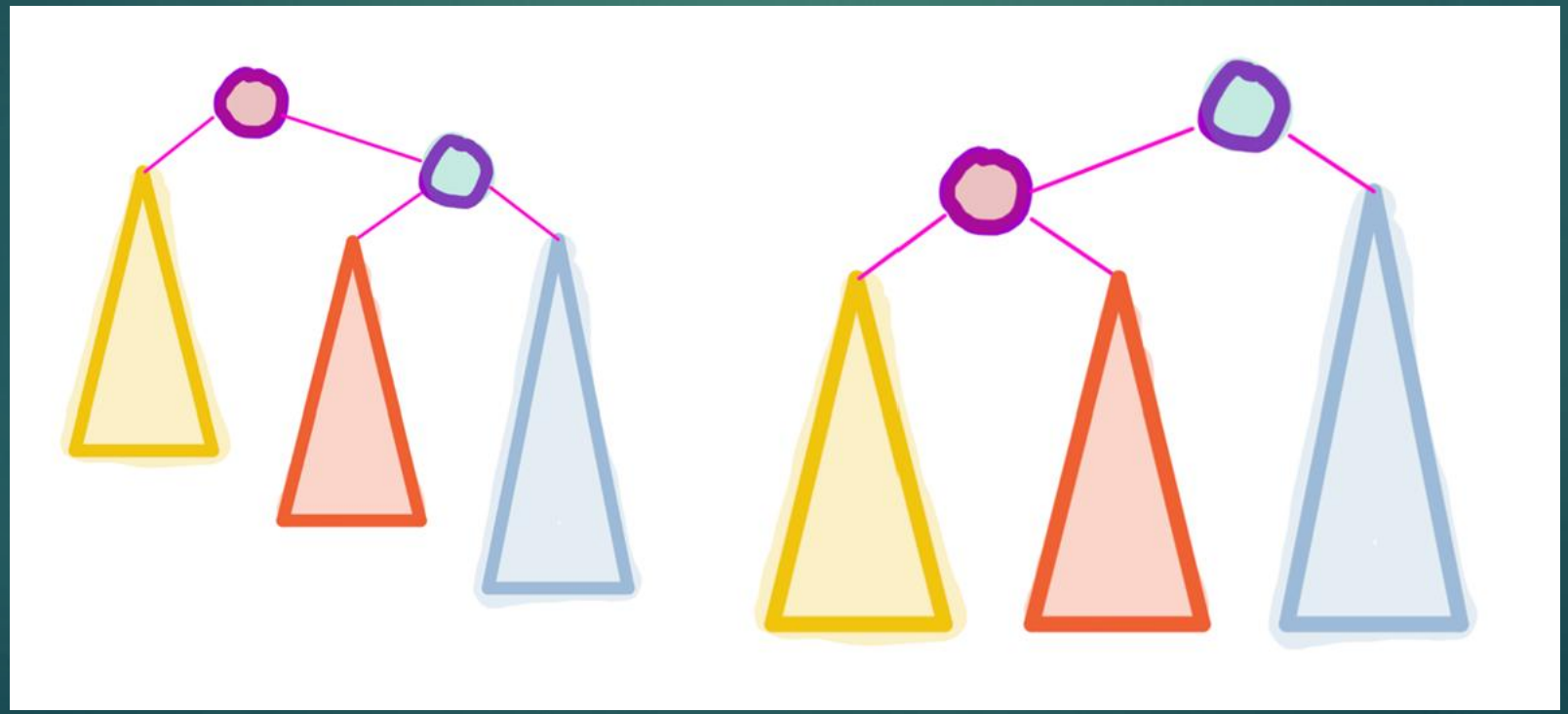
- Idea (balancing): try to keep each layer of the tree quite "full" and minimize tree height
  - It turns out we can maintain O(log n) tree height
- Many types of balanced BST, such as:
  - Red-black tree (STL implementation)
  - AVL tree (Relatively easy to implement)
  - Splay tree
  - Treap
  - Scapegoat tree
- We will talk about AVL trees

AVL tree

- Invariant: for any node, |right subtree height – left subtree height| < 2
- When is the invariant rule broken?
  - Insertions
  - Deletions
- If the invariant rule is broken, what is the height difference?
  - Always 2
- How to rebalance height difference 2?
  - 4 types of rotations
    - Left (L)
    - Right (R)
    - Left-right (LR)
    - Right-left (RL)

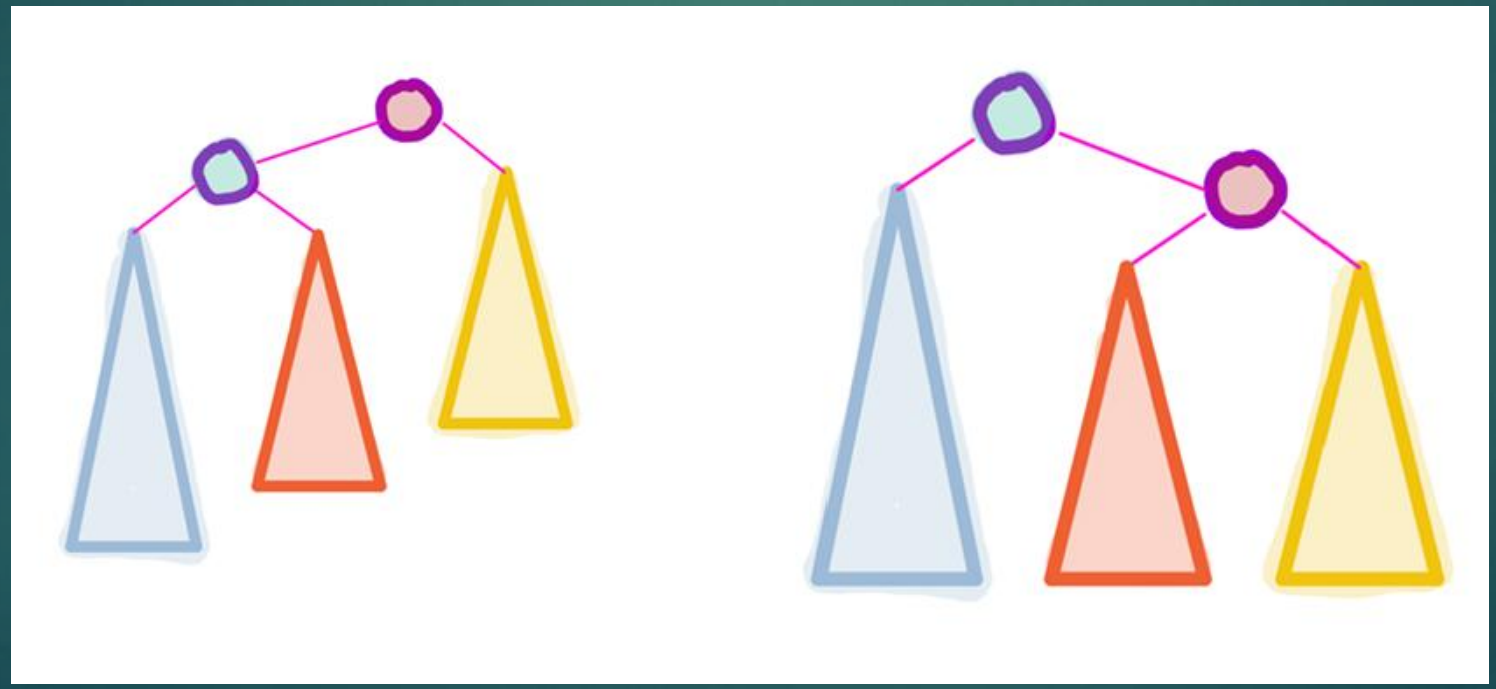
Left (L) Rotations

- Used after insertion to right subtree of right child
  - Height difference is +2 and height difference at right child is +1



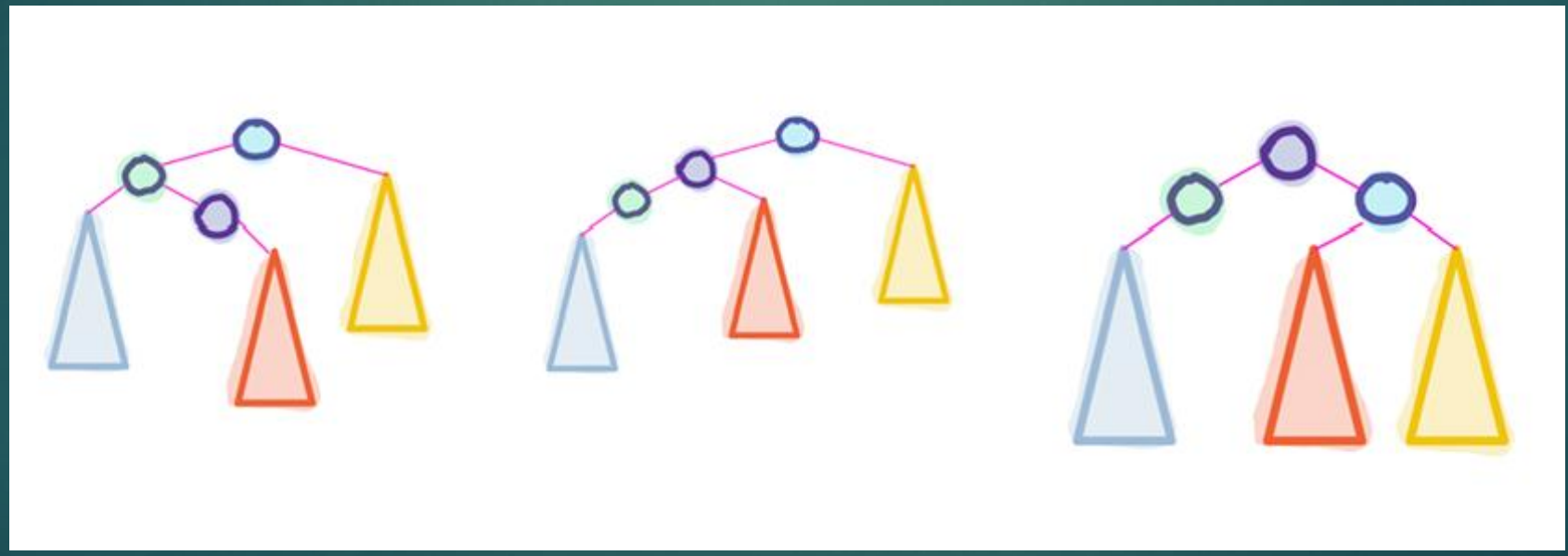
Right (R) Rotations

- Used after insertion to right subtree of right child
  - Height difference is -2 and height difference at left child is -1



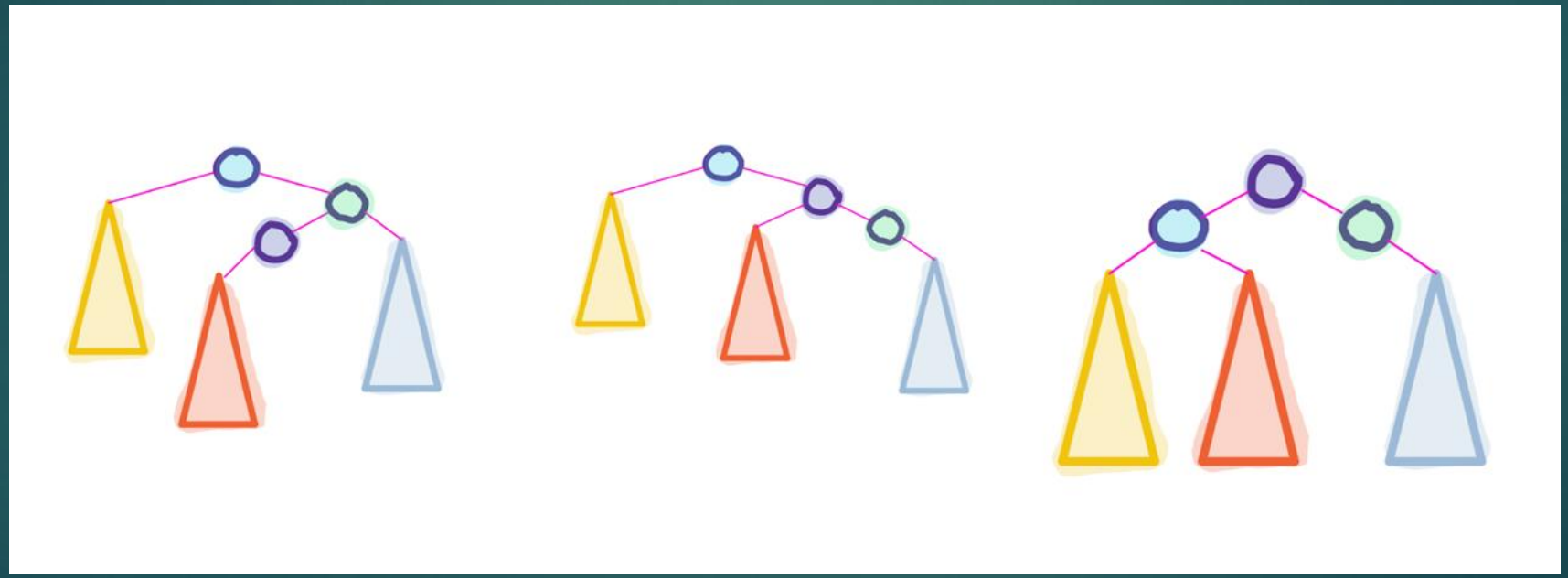
Left-right (LR) Rotations

- Used after insertion to right subtree of right child
  - Height difference is -2 and height difference at left child is +1



Right-left (RL) Rotations

- Used after insertion to right subtree of right child
  - Height difference is +2 and height difference at right child is -1



Rotation properties

- All rotations are local
  - Subtrees are not impacted
- All rotations are constant time (just pointer manipulation)
- BST property maintained

Time complexities

Operation	Runtime
Insert	O(log n)
Erase	O(log n)
Find	O(log n)
Begin	O(log n) *
Lower bound	O(log n)
Size	O(1)

* Could be implemented as O(1) for frequent access to begin()

Some things to note

- std::map and std::multimap are also implemented as a balanced BST (each node has a key-value pair and compared by key)
- std::set lacks rank() operation.
  - Rank(x) is the x^{th} smallest value of the set
- Be careful incrementing set iterator.
  - Each it++ takes O(log n) time worst case
  - But STL implementation gives O(1) time amortized after traversing entire tree.
    - `for (auto it = s.begin(); it != s.end(); it++) {/* constant work */}`

Use Cases of std::set/std::map

- Maintaining unique elements (since std::set has no duplicates)
  - Especially useful if we are maintaining elements of custom data type that is easier to compare than hash
- Checking existence
  - Always consider:
    - Do I need elements to be sorted? (consider unordered set/map)
    - What is the range of possible values? (consider array)
- Maintaining smallest and largest elements
- Iterating in order

Use Cases of std::set/std::map in HFT

- In general, std::set should be avoided in low latency systems
  - Each insert requires heap allocation (slow)
  - O(log n) operations may be slow compared to array + caching
- Used in snapshot synthesizer in trading system (more in week 6)
  - Used to recover missed market updates from exchange due to UDP
  - Receives state of order book and new market updates
  - Stores it in std::map (mapping sequence number to market update information)
  - Sequence numbers need to be sorted to find start/end of snapshot and gaps
  - Not so latency sensitive since this is a rare occurrence and trading is paused during recovery

Exercises

- Compare set and vector
- Compare BST and balanced BST
- Implement BST recursive functions iteratively and compare speed
- Implement std::multiset
- Implement std::map
- Implement set::rank (hint: maintain subtree sizes)
- Implement AVL tree (hint: maintain node height and implement left/right rotation functions)
- Implement red-black tree

Questions?